Multi-Agent Requirement Conflict Detection via Knowledge-Guided Consistency Analysis

Anonymous Author(s)

Abstract

Natural language requirements often contain semantic conflicts, logical contradictions, inconsistent constraints, role-level disagreements, and implicit incompatibilities. This paper studies requirement conflict detection as a focused software engineering problem and proposes a knowledge-guided multi-agent workflow for consistency verification. The intended approach combines collaborative requirement analysis, constraint-aware conflict identification, explanation-oriented validation, and human-in-the-loop refinement. Rather than presenting a general requirement platform, the paper is organized around one research question: how can intelligent agents and requirement knowledge support reliable detection and analysis of conflicts in natural language requirements?

Keywords

requirements engineering, requirement conflict detection, requirement consistency verification, multi-agent systems, knowledge-guided analysis, human-in-the-loop verification

1 Introduction

1.1 Background and Motivation

Requirement consistency is a central quality concern in requirements engineering. Natural language requirements are readable for stakeholders, but they can also encode semantic conflicts, logical contradictions, incompatible constraints, and conflicting role expectations. These problems are especially costly when inconsistent requirements are propagated into design, implementation, and testing.

Recent work has emphasized requirements as a primary interface for AI-assisted software engineering [2, 3]. At the same time, controlled natural language research shows that reducing ambiguity while preserving stakeholder readability remains difficult [1]. This motivates a focused study of requirement consistency verification for natural language requirements.

1.2 Challenges of Requirement Conflict Detection

Requirement conflicts are rarely limited to explicit textual contradictions. They may appear as implicit incompatibilities between constraints, mismatches between user roles and permissions, inconsistent temporal ordering, or semantic conflicts across requirements written in different styles.

This paper focuses on the following research problem:

> How can multi-agent collaboration and knowledge-guided consistency analysis support dynamic detection, explanation, and refinement of conflicts in natural language requirements?

1.3 Multi-Agent Requirement Verification

A single analyzer may miss conflicts that require different reasoning perspectives. We therefore organize requirement verification as a lightweight collaboration among analysis, validation, and review roles. The goal is not to build a large agent platform, but to structure conflict detection as a repeatable workflow that checks requirement statements against domain knowledge, constraint rules, and stakeholder feedback.

1.4 Contributions

This paper is organized around the following contributions:

- A research problem formulation for multi-agent requirement conflict detection via knowledge-guided consistency analysis.
- A focused workflow for identifying functional, constraint, role, and temporal conflicts in natural language requirements.
- A knowledge-guided verification method that combines requirement knowledge, constraint rules, semantic patterns, and prompt strategies.
- An experimental plan for evaluating conflict detection effectiveness, consistency verification accuracy, false positives, and human verification effort.

2 Related Work

2.1 Requirement Conflict Detection

This line of work studies how inconsistent or incompatible requirements can be identified before they affect downstream development. The discussion will focus on conflict types, detection assumptions, and the extent to which existing methods handle implicit conflicts in natural language specifications.

2.2 Requirement Consistency Analysis and Validation

Requirement consistency analysis examines whether requirement statements can hold together under shared system goals, constraints, and stakeholder expectations. Requirement validation complements this by checking whether reported conflicts are meaningful to stakeholders and actionable for requirement refinement.

2.3 LLM-based Requirement Analysis

LLM-based requirement analysis has been used for requirement extraction, classification, summarization, and validation. This paper only considers such work where it is directly relevant to requirement consistency verification and conflict detection, rather than general language model capabilities.

2.4 Knowledge-Guided Requirement Verification

Knowledge-guided verification uses domain concepts, requirement patterns, constraint rules, and consistency criteria to reduce purely text-similarity-based judgments. This direction is relevant because many conflicts depend on unstated assumptions or domain-specific constraints.

2.5 Multi-Agent Collaboration

Multi-agent collaboration can decompose requirement verification into complementary roles, such as analysis, validation, and review. The related work discussion will focus on how role separation can improve conflict detection reliability without turning the paper into a general agent platform study.

3 Methodology

3.1 Framework Overview

The proposed method treats requirement conflict detection as a consistency verification workflow. Given a set of natural language requirements, the workflow extracts candidate constraints, compares them against requirement knowledge, detects potential conflicts, generates explanations, and requests human refinement when automated evidence is insufficient.

3.2 Multi-Agent Conflict Analysis Workflow

3.2.1 Analyst Agent. The Analyst Agent decomposes each requirement into intent, actors, actions, conditions, constraints, and expected outcomes. It also groups requirements that are likely to interact.

3.2.2 Validator Agent. The Validator Agent checks the candidate interactions against consistency criteria, constraint rules, and known semantic conflict patterns. It produces conflict candidates with supporting evidence.

3.2.3 Reviewer Agent. The Reviewer Agent examines whether a reported conflict is sufficiently grounded, whether the explanation is understandable, and whether the case should be sent to a human stakeholder for refinement.

3.3 Requirement Conflict Identification

3.3.1 Functional Conflict. Functional conflicts occur when two requirements prescribe incompatible system behaviors for the same condition, input, actor, or business goal.

3.3.2 Constraint Conflict. Constraint conflicts occur when non-functional or business constraints cannot be satisfied together, such as mutually incompatible limits on time, access, resource usage, or policy compliance.

3.3.3 Role Conflict. Role conflicts occur when requirements assign inconsistent permissions, responsibilities, or restrictions to the same stakeholder role or to interacting roles.

3.3.4 Temporal Conflict. Temporal conflicts occur when requirements impose incompatible ordering, timing, state transition, or lifecycle constraints.

3.4 Knowledge-Guided Consistency Analysis

3.4.1 Requirement Knowledge. Requirement knowledge captures domain concepts, stakeholder roles, requirement attributes, and consistency criteria that are needed to interpret natural language requirements.

3.4.2 Constraint Rules. Constraint rules encode checkable consistency conditions, such as mutually exclusive permissions, incompatible numeric bounds, mandatory ordering, and dependency constraints.

3.4.3 Semantic Patterns. Patterns capture common conflicts, including negation, exception, role permission, and condition outcome mismatches.

3.4.4 Prompt Strategies. Prompt strategies guide each agent to produce structured evidence rather than ungrounded judgments. Prompts should require conflict type, involved requirements, consistency rule, explanation, confidence, and suggested human review action.

3.5 Requirement Verification and Refinement

3.5.1 Conflict Detection. The workflow detects candidate conflicts by comparing requirement pairs or groups under the selected knowledge and constraint rules.

3.5.2 Conflict Explanation. Each reported conflict should include the conflicting requirement statements, the violated consistency criterion, and a concise natural language explanation.

3.5.3 Conflict Resolution Suggestions. The method may suggest refinement directions, such as relaxing a constraint, clarifying an actor, adding an exception condition, or splitting a requirement. Suggestions are treated as decision support rather than automatic requirement rewriting.

3.6 Human-in-the-Loop Interaction

Human-in-the-loop interaction is used when the agents cannot determine whether a case is a true conflict or an acceptable domain-specific exception. Human feedback is used to confirm conflicts, reject false positives, and refine the knowledge or constraint rules used in later verification.

4 Experiments

4.1 Research Questions

The experiments evaluate whether the proposed workflow improves requirement conflict detection and consistency verification.

- **RQ1:** How accurately does the workflow detect conflicts in natural language requirements?
- **RQ2:** How much does knowledge-guided analysis improve consistency verification?
- **RQ3:** How does multi-agent collaboration affect false positives and explanation quality?
- **RQ4:** To what extent can human-in-the-loop refinement reduce manual verification effort?

4.2 Experimental Setup

The evaluation will compare the proposed workflow with simpler requirement checking settings under the same input requirements and annotation criteria. The setup should specify agent prompts, knowledge sources, conflict taxonomy, annotation protocol, and human verification procedure.

4.3 Datasets

The study will use requirement sets containing manually annotated conflicts and non-conflicts. Dataset reporting should include the application domain, number of requirements, conflict categories, annotation process, and inter-annotator agreement where applicable.

4.4 Baselines

Baselines should be selected to isolate the effect of collaboration and knowledge guidance:

- A single-agent requirement conflict checker.
- A requirement checker without external requirement knowledge.
- A rule-only consistency checker where applicable.

4.5 Evaluation Metrics

4.5.1 Conflict Detection Accuracy. Conflict detection accuracy measures whether the workflow correctly distinguishes conflicting and non-conflicting requirement cases.

4.5.2 Consistency Verification Accuracy. Consistency verification accuracy measures whether detected conflicts are assigned to the correct consistency issue and conflict category.

4.5.3 False Positive Rate. False positive rate measures how often the workflow reports conflicts that human annotators consider acceptable or non-conflicting.

4.5.4 Human Verification Reduction. Human verification reduction measures the decrease in manual review effort after agent-based filtering, explanation, and refinement.

4.6 Main Results

This subsection will report the main quantitative results once the annotated requirement data and baselines are finalized. No synthetic performance numbers are introduced in the paper structure.

4.7 Ablation Study

The ablation study should evaluate the contribution of major components, such as knowledge guidance, constraint rules, role-separated agents, and human feedback.

4.8 Case Study

The case study should present a small requirement set with a representative conflict, the workflow output, the explanation, and the human refinement decision.

5 Threats to Validity

5.1 Internal Validity

Internal validity threats include prompt sensitivity, inconsistent human annotations, and possible bias introduced by the selected requirement examples. The study should mitigate these threats through fixed evaluation prompts, clear annotation guidelines, and independent review of conflict labels.

5.2 External Validity

External validity threats arise because requirement conflicts vary across domains, stakeholder groups, and specification styles. The evaluation should therefore report the domains and requirement types covered by the dataset and avoid overgeneralizing from a narrow sample.

5.3 Construct Validity

Construct validity threats concern whether the selected metrics capture the intended notions of conflict detection and consistency verification. Accuracy and false positive rate should be complemented by explanation quality and human verification effort to better reflect practical usefulness.

6 Conclusion

This paper studies multi-agent requirement conflict detection as a focused problem in intelligent requirements engineering. The proposed structure centers on requirement consistency verification, knowledge-guided conflict analysis, dynamic conflict detection, and human-in-the-loop refinement. The paper is intended to evaluate whether a lightweight collaboration among analysis, validation, and review roles can improve the detection and explanation of semantic, logical, constraint, role, and temporal conflicts in natural language requirements.

References

[1] Ikram Darif, Ghizlane El Boussaidi, Segla Kpodjedo, and Cristiano Politowski. 2026. Controlled Natural Language for Requirements Specification: A Systematic Literature Review. *ACM Comput. Surv.* 58, 7 (Jan. 2026), 175:1–175:36. doi:10.1145/3778169

[2] Hojae Han, Jaejin Kim, Jaeseok Yoo, Youngwon Lee, and Seung-won Hwang. 2024. ArchCode: Incorporating Software Requirements in Code Generation with Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 13520–13552.

[3] Diana Robinson, Christian Cabrera, Andrew D. Gordon, Neil D. Lawrence, and Lars Mennen. 2025. Requirements Are All You Need: The Final Frontier for End-User Software Engineering. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (May 2025), 141:1–141:22. doi:10.1145/3708524

A Appendix

The appendix can include supplementary materials such as the conflict annotation guideline, agent prompt templates, consistency rule examples, and additional case study details. It should not introduce new claims that are unsupported by the main evaluation.